Rambus

Security IP
EIP-130

Root of Trust engine
Driver v3.1

User Guide

Document Revision: A
Document Date: 2022-02-09
Document Number: 007-130310-301
Document Status: Accepted

© Rambus Inc. • rambus.com

Copyright 2015-2022 Rambus Inc. This document contains information which is proprietary and confidential, and which is protected under patents, copyrights, and/or other IP rights of Rambus Inc. If you are not the intended recipient of this material, please destroy this document and inform Rambus at +1 408 463 8000 or sipsupport@rambus.com immediately.

Rambus Inc. Corporate Headquarters
4453 North First Street, Suite 100
San Jose, CA 95134
Phone: +1 408-462-8000
Website : https://www.rambus.com/
Contact : sipsupport@rambus.com

Rambus ROTW Holding B.V.
Boxtelseweg 26A
5261 NE Vught
The Netherlands
Phone: +31-73-6581900

Table of Contents

Table of Contents 3
List of Tables 5
List of Figures 5
Document Revision History 6
1 Introduction 7
1.1 Purpose 7
1.2 Scope 7
1.3 Related Documents 7
1.4 Target Audience 7
2 Main Components Overview 8
2.1 VAL Driver: VaultIP Exchange 9
3 Driver Customization Instructions 11
3.1 Multiple CPU use case customization 11
3.1.1 AMP use case 11
3.1.2 SMP use case 11
3.2 Adapter customization 11
3.2.1 Driver hardware platform initialization 11
3.2.2 Driver entry and exit points 12
3.2.3 Execution context synchronization 12
3.2.4 User-space to kernel-space communication 12
3.2.5 Firmware download 13
3.2.6 Firmware update or recovery 13
3.3 Driver Framework Customization 13
3.3.1 DMA-safe buffer allocation for kernel-space drivers 15
3.4 Kernel Support Driver 15
3.4.1 DMA-safe buffer allocation for user-space drivers 15
3.5 Customization for ARM-based platform 15
3.5.1 Device registration in Linux kernel 15
3.5.2 DMA buffers allocation and mapping to user-space 16
4 Driver Deployments 17
4.1 Kernel-space Driver 17
4.2 User-space Driver 17

5 Application Support 18
5.1 Single Application Support 18
5.2 Multiple Application Support 18
6 Driver Build and Test Instructions 19
6.1 Build Instructions for a single configuration 19
6.2 Test Instructions 20
6.3 Build and Test Instructions for Multiple Configurations 20
7 Driver API 21
7.1 Combined Driver Initialization API 21
7.2 Global Control Driver Initialization API 21
7.3 VAL Driver Initialization API 21
7.4 Global Control API 21
7.5 VAL API 21
7.6 Memory Access API 21
8 Data Structures 22
8.1 Dynamic Resource (DMA) Data Structures 22
8.2 EIP-130 Tokens and Data Buffers 22
9 Driver Configuration Parameters 23
9.1 Top-level Driver configuration parameters 23
9.2 Top-level Adapter configuration parameters 24
9.3 Top-level Driver Framework (HWPAL) configuration parameters 26
9.3.1 Default Driver Configuration 28
9.4 Driver Traces (Logging) Configuration 28

List of Tables

Table 1 Driver build, Supported API and Deployment 18

List of Figures

Figure 1 VAL Driver: Deployment 9
Figure 2 VAL Driver: Overview of VEX token serialization mechanism 10

Document Revision History

Doc Rev	Page(s) Section(s)	Date (Y-M-D)	Author	Purpose of Revision
A	All	2022-02-09	MHO	• First release for v3.1, based on v3.0 RevC.

1 Introduction

1.1 Purpose

This *EIP-130* Driver provides control functionality for the *EIP-130* hardware (also referred to as *Root of Trust / RT or VaultIP*). The Driver is implemented as a software component running on the host CPU. The *EIP-130* hardware protects valuable assets (Security Critical Parameters), such as secret keys for security sensitive applications. Additionally, the *EIP-130* embeds hardware implementations of cryptographic algorithms supporting a variety of operations and modes.

Refer to the DDK Release Notes for the information about the *EIP-130* hardware versions and firmware versions supported by the Driver.

Newer *EIP-130* hardware or firmware may be supported by this Driver. For more information contact Rambus technical support.

The Driver functionality can be built for different deployments and configurations. Moreover, the Driver can be built to support a certain subset of the APIs.

The Kernel Support Driver is an internal Driver, its API is not exposed to the applications and its details are out of the scope of this document.

1.2 Scope

This document:

- provides a high-level overview of Driver components
- lists instructions on how to build the Driver
- provides references to the API provided by the Driver
- describes important configuration parameters of the Driver
- provides references to the other documents useful for understanding the Driver in more detail

1.3 Related Documents

Ref	Document	Doc. Number
[1]	Security IP Driver Framework Porting Guide	007-900450 -304
[2]	EIP-130 DDK Test Specification	007-130310-310
[3]	EIP-130 Driver Software Block Diagram	N/A
[4]	EIP-130 Driver Library User Guide	007-130310-307
[5]	Security IP-201 Driver Library User Guide	007-201320-307
[6]	README Regression System	N/A
[7]	README Test Tool VAL	N/A

This information is correct at the time of document release. Rambus reserves the right to update the related documents without updating this document. Please contact Rambus for the latest document revisions.

The preferred method for getting technical support is to use our online support system at https://sipsupport.rambus.com. If you do not have an account yet for this system, please contact Rambus technical support (sipsupport@rambus.com).

1.4 Target Audience

This document is intended for software developers who want to use the *EIP-130* Driver.

2 Main Components Overview

The *EIP-130* Driver is composed of components listed below. It can be built as a Linux kernel module and needs to be loaded as a module in the Linux kernel. Alternatively, the Driver can be built as a Linux user-space statically linkable library that can be linked with a Linux user-space application. The Driver provides several APIs (public interfaces). The most important API is the VAL API. The other APIs can be divided in two groups, one to initialize or de-initialize a driver type and one intended for internal use only (for example the VEX API). The distinguished driver types are:

- Combined Driver Combines the next two driver types:
- Global Control Driver Provides the Global Control API.
- VAL Driver Provides the VAL API and VEX.

Component	Remarks
Config	The cs_*.h files that hold the generic configuration settings.
Examples/Driver/build	Makefile and configuration files to build the Combined Driver.
Examples/Driver_GC/build	Makefile and configuration files to build the Global Control (GC) Driver.
Examples/Driver_VAL/build	Makefile and configuration files to build the *VaultIP* Abstraction Layer (VAL) Driver.
Integration/Adapter_DriverInit/src	The driver initializes and de- initializes implementation for the various Adapters.
Integration/Adapter_DriverInit/incl	The API definition for the driver initialization and de-initialization. api_driver_init.h: Combined Driver API implementation api_driver_gc_init.h: Global Control Driver API implementation api_driver_val_init.h: VAL Driver API implementation ***Note:*** Please refer to the included header file for API details.
Integration/Adapter_Generic/src	Generic implementation that the various Adapters can use.
Integration/Adapter_GlobalControl/src	The Global Control Adapter provides the implementation of the Global Control API.
Integration/Adapter_GlobalControl/incl	The Global Control Adapter API definition. api_global_eip130.h: API exposed by Driver to control Global Control specific functionality. Part of the Global Control API. api_memxs.h: MemXS API for accessing the internal EIP HW registers and memory, mostly designed for use from test tools.
Integration/Adapter_Interrupts/src	Implementation of interrupt handler registration.
Integration/Adapter_VAL/src	The VaultIP Abstraction Layer (VAL) Adapter provides the implementation of the VAL API.
Integration/Adapter_VAL/incl	The VAL API definition. api_val.h: The main entry for the VAL API exposed by the Driver. Please refer to included header file for details of the API.
Integration/Adapter_VEX/src	The VaultIP Exchange (VEX) Adapter provides the implementation of the VEX API. Note that the VEX API is an internal API and therefore the API definition can be found here as well.

The Driver depends on other software components, which are included in the Kit directory (see *Driver Software Block Diagram* [3] for a diagram showing these components):

- The Log component for outputting error/warning/informational messages.
- The *EIP-130* Driver Library, for low-level access to the *EIP-130 Root of Trust engine*. This library contains the Register, Mailbox and Firmware Control, and additionally the Token Helper functionality.
- The *Security IP-201* Driver Library, for low-level access to the *Security IP-201* AIC interrupt controller *(also referred to as EIP-201)*, which is part of the *EIP-130* hardware. This library contains register-level access functions.
- The Driver Framework, layer for portability across different hardware platforms and OSs.

Furthermore, the Driver depends on the following software components under the Integration directory:

- The Driver Framework implementation, consisting of a generic part and several parts that depend on the driver deployment type.

In addition, the Driver uses several more software components when it is deployed in the Linux user-space and is accessed by user mode applications. These are included from the Integration directory.

- The VEX Proxy Stub is a component that mirrors the VEX API in user-space, which resides in kernel-space as part the Combined Driver or VAL Driver.
- UMDevXS (Kernel Support Driver for the user-space device access with its user-space component), which provides the required Linux kernel services to the Combined Driver deployed in the user-space.

2.1 VAL Driver: VaultIP Exchange

The VAL Driver has a component called VEX, which implements the *VaultIP* Exchange. With the *VaultIP* Exchange is meant the token communication, which is serialized, between the User Space and Kernel Space applications and the *EIP-130* hardware. For Linux, a simplified deployment is shown in the next figure.

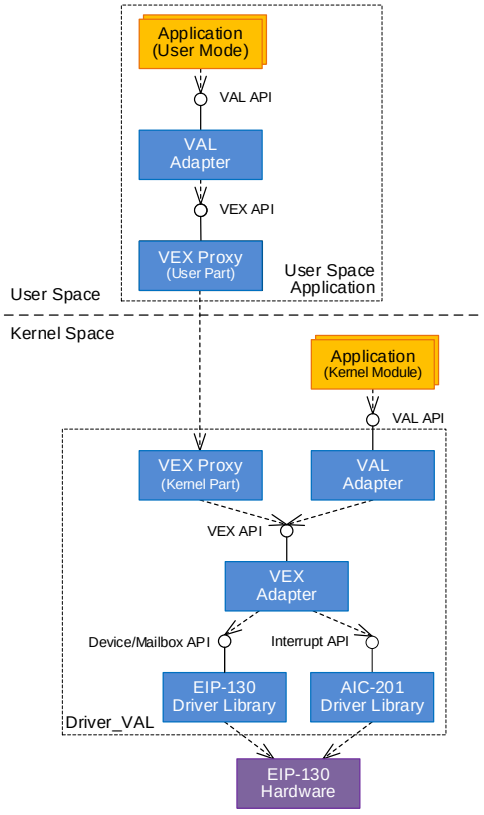


Figure 1 VAL Driver: Deployment

The token serialization is handled in the VEX Adapter. When the VAL Adapter or VEX Proxy (Kernel Part) calls the `vex_LogicalToken()` function the submitted (logical) command token is put in a Command FIFO (queue). The entries in this FIFO queue are processed one by one in a `VEX_HndToken` thread that converts the logical command token to a physical one. The physical command token is then sent to the *EIP-130* hardware. When the *EIP-130* hardware has the result available, the physical result token is converted into the logical result token and the initial command entry is updated and moved to the Result list. The initial submitter will get the result form the Result list. Note that `VEX_HndToken` thread is notified when an entry is placed in an empty Command FIFO. An application is notified when the result is placed in the Result list.

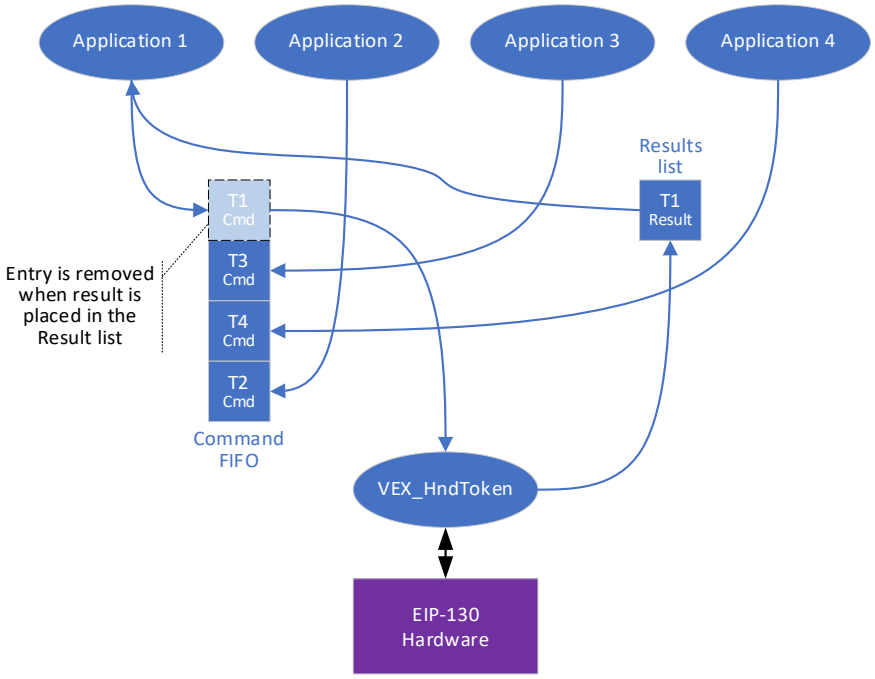


Figure 2 VAL Driver: Overview of VEX token serialization mechanism

3 Driver Customization Instructions

This chapter describes several important customization areas in the *EIP-130* Driver. The *EIP-130* Driver is delivered as an Example Driver that must be customized for a host chip or chipset with the integrated *EIP-130* hardware. Such customization may include the customization of the Driver Framework implementation specific to the chip or operating system. For more information, refer to the *Driver Framework Porting Guide* [1].

The *EIP-130* Driver is delivered with example code either for the Xilinx Virtex6 PCI-Express FPGA board, Xilinx Zynq-7000 ZC706 FPGA board or ARM Juno FPGA board with a Logic Tile FPGA programmed with the *EIP-130* logic. The Xilinx Zynq-7000 ZC706 board has been used for the Example Driver verification. The Xilinx Virtex6 FPGA PCIe board with an x86 host and ARM Juno board with a Logic Tile FPGA have an ARM CPU running Linux with the *EIP-130* directly attached to the AXI system bus and have been used for sample testing of the Example Driver. The Xilinx Zynq-7000 ZC706 board has also an option of using the MicroBlaze microcontroller inside the FPGA logic.

The following sections describe how such customization work can be done in these areas.

3.1 Multiple CPU use case customization

3.1.1 AMP use case

If multiple CPUs must be used independently (this means each CPU has its own independent OS) then each CPU must perform mailbox I/O to communicate with *EIP-130*. However, in this use case, only one Global Control Driver instance may be used, and that instance must run on the master CPU and has access to the entire *EIP-130* functionality. On all CPUs, the VAL Driver must be used for the mailbox I/O.

Note: The Combined Driver cannot be used for the AMP use case.

3.1.2 SMP use case

If all the CPUs are used as one processor pool under control of a single OS, then there are two options. Either use the Combined Driver (combines the Global Control and the VAL driver in one), or still use separate Global Control and VAL Driver. The *EIP-130* mailboxes can be assigned per application so that each application can communicate with *EIP-130* independently from other applications.

3.2 Adapter customization

The following functionality may need to be customized in the Adapter layer.

3.2.1 Driver hardware platform initialization

FPGA or other hardware platform specific initialization and reset functionality:

- Global Control Driver initialization
  - `Integration/Adapter_DriverInit/src/$HOST_HW_PLATFORM/adapter_global_control_init.c`
- VAL Driver initialization
  - `Integration/Adapter_DriverInit/src/$HOST_HW_PLATFORM/adapter_init_val.c`
- Combined Driver initialization
  - `Integration/Adapter_DriverInit/src/$HOST_HW_PLATFORM/adapter_init.c`

3.2.2 Driver entry and exit points

The platform specific code for the driver first and the last functions to be called by the system depending on the deployment is as follows:

- Linux kernel-space (`Integration/Adapter_DriverInit/src/lkm/`):
  - Global Control Driver:
    - `adapter_global_control_init_ext.h`
  - VAL Driver:
    - `adapter_val_init_ext.h`
    - `adapter_val_lkm_ext.h`
  - Combined Driver:
    - `adapter_init_ext.h`
- Linux user-space or Microcontroller (`Integration/Adapter_DriverInit/src/um/`):
  - Global Control Driver:
    - `adapter_global_control_init_ext.h`
  - VAL Driver:
    - `adapter_val_init_ext.h`
  - Combined Driver:
    - `adapter_init_ext.h`

3.2.3 Execution context synchronization

A CPU code execution context details may vary per platform, examples are threads, top and bottom halves interrupt handlers. Different types of execution contexts may require different synchronization and delayed wait methods.

Reading current running process ID:

- `Integration/Adapter_Generic/src/mb_freertos/adapter_getpid.c`

Putting current running execution context to a sleep:

- `Integration/Adapter_Generic/src/mb_freertos/adapter_sleep.c`

Locking functionality to synchronize currently running execution contexts:

- `Integration/Adapter_Generic/src/mb_freertos/adapter_lock.c`

Condition wait functionality for signaling waiting execution contexts from another:

- `Integration/Adapter_Generic/src/lkm/adapter_event.c`
- `Integration/Adapter_Generic/src/um/adapter_event.c`

3.2.4 User-space to kernel-space communication

Some general-purpose CPUs such as x86 and arm have a memory management unit (MMU) that is used to separate memory domains where the application and system-critical code can be executed. The operating system, such as Linux, also supports the MMU and provides the address space separation, for example Linux kernel-space and user-space. Microcontrollers typically do not have an MMU and consequently do not have the address space separation. In case of a CPU with MMU a driver controlling a hardware block would need to have some of it functionality execute in the privileged address space, such as Linux kernel-space, for example for mapping hardware resources into the address space, executing low-level interrupt code and allocating memory for DMA operations, see section 3.4. The driver may be deployed partly in one address space, closer to hardware in a (Linux kernel) privileged space, and partly in another, closer to application code in a non-privileged (Linux user) space.

The provided example drivers implement two methods for the Linux user-space to kernel-space communication. The first method is via the Kernel Support Driver that provides required Linux kernel service to the user-space deployed driver, such as hardware resources mapping into user-space, interrupt handling

and dispatching to user-space and DMA buffer allocation and mapping to user-space. Refer to section 3.4 for the customization details for this driver. The second method is to proxy the driver interface deployed in the kernel-space to its driver counterpart deployed in the user-space. Note that in this method not the high-level driver VAL API is proxied, but the internal driver VEX interface. The VEX Proxy Stub example implementation can be found in the `Integration/VEX_Proxy_Stub` folder.

The Adapter layer provides example code for the Linux kernel-space and user-space as well as for the MicroBlaze microcontroller where the entire code stack executes just in one address space.

The Adapter Linux kernel-space specific code can be found in the `Integration/Adapter_*/src/lkm` folders. The Adapter Linux user-space specific code can be found in the `Integration/Adapter_*/src/um` folders. In addition, the Adapter microcontroller specific code can be found in the `Integration/Adapter_*/src/mb_freertos` folders.

3.2.5 Firmware download

The *EIP-130* contains a programmable engine, which requires firmware to run. This firmware can either be in ROM or RAM. Only in case of RAM, the driver requires the firmware download functionality. The required firmware image (a binary file, which has normally the name `firmware_eip130.sbi`) in that case is generated with a tool that is provided with the Firmware package. The Adapter firmware download API is defined in the file `Integration/Adapter_Generic/src/adapter_firmware.h`. An implementation of this API receives an image name as input, and it shall provide a reference to the firmware image in an array of 32-bit words as output. The image name is a null-terminated string, and it is typically a file name.

There are three implementations of the firmware download API:

- For Linux kernel-space deployment, the implementation is in `Integration/SLAD_Adapter/src/lkm/adapter_firmware.c`. It uses the Linux kernel firmware download API to retrieve the firmware file stored in the `/lib/firmware` directory.
- For Linux user-space deployment, the implementation is in `Integration/SLAD_Adapter/src/um/adapter_firmware.c`. It uses the standard C library to load files into memory. This firmware file is stored in the `/lib/firmware` directory by default, but the path name can be configured at compile time via the ADAPTER_FIRMWARE_PATH_PREFIX parameter.
- For microcontroller deployment without file system support, the implementation is in `Integration/SLAD_Adapter/src/mb_freertos/adapter_firmware.c`. It uses the statically linked firmware image blob file. The blob object file for linking can be generated using the Linux `objcopy` tool.

3.2.6 Firmware update or recovery

In case of failure, the active firmware image can be replaced by a recovery image. The Host must decide when to switch to a different firmware image. On file-based systems this can be done by simply replacing the active firmware image with another and then restarting the *EIP-130* . On microcontrollers, this can be done by pointing the Firmware memory address to the new image. The underlying mechanisms for this are customer dependent. Please contact Rambus support when more information is required.

3.3 Driver Framework Customization

The Driver Framework implementation module implements the Device API (see also the *Driver Framework Porting Guide* [1] for details) such as register and internal memory read and write functions specific to the host chip hardware with the integrated *EIP-130* hardware. This component also implements the DMAResource API.

The Driver Framework component contains an example code for the *EIP-130* hardware access via:

- PCI (or PCI-Express) bus implemented for the Xilinx Virtex6 FPGA PCIe platform.
- Linux OpenFirmware API for the situations when the *EIP-130* hardware is directly attached to the system bus, which is the case for the ARM Juno board with a Logic Tile FPGA, as well as for a microcontroller with an RTOS deployment, which is the case for the MicroBlaze FreeRTOS Xilinx Zynq ZC706 FPGA board.

The bus attachment type of the *EIP-130* hardware determines the implementation of the Driver Framework Device API, but not the DMAResource API.

If the *EIP-130* hardware is integrated in a host chip with different bus architecture, or the Linux kernel interfaces to the bus in a different way, then this implementation must be customized.

- Interfaces:
  `Kit/DriverFramework/Device_API/incl`
  `Kit/DriverFramework/DMAResource_API/incl`
- Example implementation:
  - Generic code (common to all driver deployments)
    - `Integration/DriverFramework/src/device_generic.c`
    - `Integration/DriverFramework/src/dmares_gen.c`
    - `Integration/DriverFramework/src/dmares_hwpal.h`
    - Configuration files
      - `Config/cs_hwpal_lkm.h`
      - `Config/cs_hwpal_ext2.h`
    - FPGA board specific configuration file
      - `Config/$HOST_HW_PLATFORM/cs_hwpal_ext.h`
  - Linux kernel-space driver deployment
    - `Integration/DriverFramework/src/LKM/dmares_lkm.c`
    - For PCI-attached EIP devices
      - `Integration/DriverFramework/src/LKM/PCI/device_lkm.c`
    - For directly bus attached EIP devices
      - `Integration/DriverFramework/src/LKM/OF/device_lkm.c`
  - Linux user-space driver deployment
    - `Integration/DriverFramework/src/UMDevXS/hwpal_dmares_umdevxs.c`
    - `Integration/DriverFramework/src/UMDevXS/hwpal_device_umdevxs.c`
    - Driver Framework configuration file
      - `Config/cs_hwpal_umdevxs.h`
    - `Integration/UMDevXS/UserPart/src/umdevxsproxy.c`
    - `Integration/UMDevXS/KernelPart/src/umdevxs_*.c`
    - Kernel Support Driver configuration file
      - General configuration file
        - `Examples/Driver_KernelSupport/UMDevXS/build/cs_umdevxs.h`
      - FPGA board specific configuration file
        - `Examples/Driver_KernelSupport/UMDevXS/build/$HOST_HW_PLATFORM/cs_umdevxs_ext.h`
  - Microcontroller with an RTOS deployment
    - `Integration/DriverFramework/src/UMDevXS/hwpal_dmares_mb.c`
    - `Integration/DriverFramework/src/UMDevXS/hwpal_device_mb.c`
    - Driver Framework configuration file
      - `Config/$HOST_HW_PLATFORM/cs_hwpal_mb.h`

3.3.1 DMA-safe buffer allocation for kernel-space drivers

The kernel-space Driver Framework DMAResource API implementation can be configured for allocating DMA-safe buffers either in cached or non-cached memory (via the define `HWPAL_DMARESOURCE_ALLOC_CACHE_COHERENT`). In case of cached memory, the functionality can be further tuned via the `HWPAL_DMARESOURCE_MINIMUM_CACHE_CONTROL` configuration parameter. This driver functionality relies on the correct Linux kernel DMA API implementation for the used platform.

Note: The configuration parameters `HWPAL_DMARESOURCE_ALLOC_CACHE_COHERENT` and `HWPAL_DMARESOURCE_MINIMUM_CACHE_CONTROL` cannot be used on some platforms.

3.4 Kernel Support Driver

The Kernel Support Driver (UMDevXS API only) is required for the Linux user-space deployment of the *EIP-130* Combined Driver. The Kernel Support Driver is not required for OSs, which do not have kernel-user-space separation. This driver set can be used for the asymmetric multi-processor kernel configuration, but it also can be used for the uni-processor and symmetric multi-processor kernel configuration.

The Kernel Support Driver implementation for user-space device access (UMDevXS):

- Kernel modules `Integration/UMDevXS/KernelPart/src`
- User-space Proxy `Integration/UMDevXS/UserPart/src`

The Kernel Support Driver UMDevXS configuration files can be found here:

- General configuration file
  - `Examples/Driver_KernelSupport/UMDevXS/build/cs_umdevxs.h`
- FPGA board specific configuration file
  - `Examples/Driver_KernelSupport/UMDevXS/build/$HOST_HW_PLATFORM/cs_umdevxs_ext.h`

3.4.1 DMA-safe buffer allocation for user-space drivers

The Kernel Support Driver supports allocation of DMA-safe buffers and mapping them to the user-space via its UMDevXS SMBuf API. For the x86 and arm/arm64 host CPUs the DMA buffers will be mapped to the user-space as non-cached. For PowerPC/PowerPC64 host CPUs the DMA buffers will be mapped to the user-space as cached. See also section 3.5.2.

Kernel Support Driver UMDevXS SMBuf implementation can be found here:

- `Integration/UMDevXS/KernelPart/src/umdevxs_smbuf.c`

3.5 Customization for ARM-based platform

This section contains guidelines for the driver customization for an ARM platform.

3.5.1 Device registration in Linux kernel

In case the *EIP-130* hardware is integrated in a platform as a non-PCI device then it will need to be made known to the Linux kernel. One method would be to use the Linux kernel `platform_device_register()` function. This function takes as an argument a pointer to the device data structure that describes the device's resources such as memory mapped I/O, device name.

For example, the ARM Juno board with a Logic Tile FPGA was used for the driver verification. The Linux kernel platform-specific code can be found in the `arch/arm64` folder. These files can be customized to register the *EIP-130* hardware with the Linux kernel so that the kernel driver can probe the device for its resources. The driver will have to register itself in the kernel using the kernel `platform_driver_register()` function using the same device name as used in the device resource data structure while registering the device.

An alternative method would be to use the Linux kernel Device Tree Structure file in the folder `arch/arm64/boot/dts` where a new record for the *EIP-130* hardware could be added. Refer to the Linux kernel documentation for details about the device tree structure files. The driver can use the Linux kernel OpenFirmware API (exported under GPL) to read the registered device resources, such as its register

mapping and interrupt assignments. An example of such device record in the `juno.dts` device tree file is provided below.

```
    EIP130@060100000 {
          #address-cells = <2>;
          #size-cells = <2>;
          compatible = "EIP130";
          interrupt-parent = <&gic>;
          interrupts = <0 61 4>;
          reg = <0x0 0x60100000 0 0x03F13FFF>;
    } ;
```

3.5.2 DMA buffers allocation and mapping to user-space

On recent ARM designs multiple memory page mappings with different attributes (for example, cached or non-cached) can cause system issues such as memory corruption. These mappings are used by the *EIP-130* hardware for the user-space deployment where the DMA buffers must be accessible directly to the user-space application and the *EIP-130* hardware. To avoid conflicting memory mappings for the DMA buffers the DMA memory in the kernel is allocated by means of the kernel DMA API `dma_alloc_coherent()` function before mapping this memory to the user-space as non-cached via the `remap_pfn_range()` function. See `umdevxs_smbuf.c` file for details. This workaround can be enabled via the `UMDEVXS_SMBUF_UNCACHED_MAPPING` and `UMDEVXS_SMBUF_CACHE_COHERENT` configuration parameters in the Kernel Support Driver and it is required for the user-space deployment only. For the kernel-space deployment, the standard cacheable DMA buffer allocation can be used.

4 Driver Deployments

The *EIP-130* Driver can be deployed in different ways:

- The Driver runs in Linux kernel-space and the applications requiring the Driver, run either in Linux kernel-space and/or Linux user-space.
- Both the Driver and the applications requiring the Driver run in the Linux kernel-space.
- Both the Driver and the applications requiring the Driver run in the Linux user-space.
- Both the Driver and the applications requiring the Driver run in the microcontroller flat memory space.

4.1 Kernel-space Driver

The kernel Driver is a Linux kernel module. There are two possible kernel drivers use cases available:

- `driver_gc_k.ko` and `driver_val_k.ko` combination. Where `driver_gc_k.ko` exports all functions of the Driver Initialization API and the Global Control API and `driver_val_k.ko` all functions of the VAL API to other kernel modules.
- `driver_vip_k.ko` that exports all functions of the Driver Initialization API, the Global Control API and the VAL API to other kernel modules.

An application can be built as a kernel module and if it is loaded after the Driver, it can use the APIs exported by the Driver(s).

Note that a user-space application can use the kernel-space driver via the VEX Proxy Stub component that provides a gateway for the VAL API from the user-space to the kernel-space. In that case, the higher-level part of VAL API is linked to Application.

4.2 User-space Driver

The *EIP-130* Driver can be compiled as a Linux user-space statically linkable library instead of a kernel module. In this case, the user-space applications must link with `libdriver_vip_u.a` and with the standard pthread library. In this deployment, the Driver provides the same APIs as in the kernel-space deployment. However, an additional component from the Integration directory is required to build the user-space Driver, which provides the required Linux kernel services to the user-space. This component is Linux_UMDevXS, which is the support kernel Driver. The user-space part of this support Driver `umdevxs_proxy.o` is part of the user-space *EIP-130* Example Driver.

This support kernel Driver can be built as the `umpci_k.ko` module for PCI and as the `umdevxs_k.ko` module for non-PCI. It requires a suitable character device node (for example, `/dev/umpci_c` for PCI and `/dev/umdevxs_c` for non-PCI), which can be created as follows:

1. Retrieve/check the major device number as follows:

```
$ cat /proc/devices |grep um
254 umpci_c (umdevxs_c)
```

2. Create the device node as follows:

```
$ mknod /dev/umpci_c c 254 0
```

or

```
$ mknod /dev/umdevxs_c c 254 0
```

5 Application Support

The application is defined as a separate memory domain such as a Linux user-space process or the entire Linux kernel-space. The *EIP-130* Driver functionality can be built to support a single application or multiple applications. The table below gives an overview of the Driver build options and the support APIs.

Table 1 Driver build, Supported API and Deployment

Driver Build	Supported API	Supported Deployment
Driver Combined	• Driver Combined Init API • Global Control API • VAL API	• User-space • Kernel-space • Single application only
Driver Global Control	• Driver Global Control Init API • Global Control API	• Kernel-space • Multiple application in combination with Driver VAL
Driver VAL	• Driver VAL Init API • VAL API	• Kernel-space • Multiple application in combination with Driver Global Control

Note that the Combined Driver cannot be used together with any of the other Drivers from this table.

5.1 Single Application Support

If only one application requires the *EIP-130* services then the simplest solution is to use the Combined Driver. Refer to the Table 1 for details about the supported APIs.

If the application runs in the user-space, then the Kernel Support Driver must be deployed next to the Combined Driver. In this case, the Combined Driver is statically linked with the application and the Kernel Support Driver runs as a kernel module. To support the Combined Driver, the Kernel Support Driver must be built using the Makefile located in the directory `Examples/Driver_KernelSupport/UMDevXS/build`. It can be convenient to use the Combined Driver in the user-space deployment for application development and debugging, for example, and later switch over to the kernel-space deployment for the final deployment in the real system.

If the application runs in the kernel-space, then the Kernel Support Driver is not required.

5.2 Multiple Application Support

For the multiple application support, the Combined Driver cannot be used, and the Global Control Driver and VAL Driver combination must be used instead. The Global Control Driver as well as the VAL Driver can be instantiated only once.

For applications to use the *EIP-130* services, the VAL driver must be loaded on each Host (one instance per Host). Both types of applications (user-mode and kernel mode) can be used at the same time. The Global Control Driver must be loaded on only one Host (the master), thus only one instance may exist system wide. The Global Control Driver must be used to control the global *EIP-130* settings and read the status information via the Global Control API.

It is also possible to use the Global Control Driver and VAL Driver for a single application instead of the Combined Driver.

6 Driver Build and Test Instructions

6.1 Build Instructions for a single configuration

If needed, set the required Driver parameters in the configuration files (`cs_*.h`), which are either in the directory `Config/` or `Examples/$DRIVER/build/` where `$DRIVER` can be set to one of the following depending on what driver needs to be built:

```
Driver
Driver_GC
Driver_VAL
```

In the provided example driver code only, the `Driver` build from all the builds listed above is used for the microcontroller deployment.

The high-level parameters (see section 9) can be modified in `cs_driver.h`. The other `cs_*.h` files contain detailed fine-tuning parameters that normally do not require user edits.

3. Make sure the Linux kernel for the host hardware platform is correctly configured. Refer to the *README Regression System* [6] for the example Linux kernel configuration files.

4. Change directory to `Examples/$DRIVER/build`, for example:

```
$ cd Examples/$DRIVER/build
```

5. Set `KERNEL_DIR` and `ARCH` in the user-editable section of Makefile or as shell variables, for example:

```
$ export KERNEL_DIR=/usr/src/linux
$ export ARCH=x86
```

or

```
$ export ARCH=arm
```

or

```
$ export ARCH=arm64
```

or

```
$ export ARCH=mb
```

6. If building for ARM or MicroBlaze, set cross compilation variable and the cross-library path variable, for example:

```
$ export CROSS_COMPILE=arm-linux-gnueabihf-
$ export CROSS_LIB_PATH=/opt/Xilinx/SDK/gnu/aarch32/lin/gcc-arm-linux-
gnueabi/lib/gcc/arm-linux-gnueabihf
```

or

```
$ export CROSS_COMPILE=aarch64-linux-gnu-
$ export CROSS_LIB_PATH=/opt/Android/toolchains/gcc-linaro-aarch64
```

or for MicroBlaze

```
$ export CROSS_COMPILE=mb-
$ export CROSS_LIB_PATH=/opt/zynq-zc706-bsp/lib
```

7. If running on a 64-bit platform, set `DDK_HOST64` to “y”, for example:

```
$ export DDK_HOST64=y
```

8. Set the HOST_HW_PLATFORM variable to the appropriate hardware platform (only one FPGA type is supported, depending on the version of the DDK), for example:

```
$ export HOST_HW_PLATFORM=FPGA_Virtex6_PCI
```

or

```
$ export HOST_HW_PLATFORM=FPGA_Zynq_ZC706
```

or

```
$ export HOST_HW_PLATFORM=FPGA_V2M_Juno
```

9. Invoke make from Examples/$DRIVER/build, for example:

```
$ make
```

This will produce both the kernel module binary (driver_vip_k.ko for the Combined Driver) and the user mode Driver in the form of a static library (libdriver_vip_u.a for the Combined Driver) in the same directory.

To use the user-space Combined Driver deployment Kernel Support Driver in the directory Examples/Driver_KernelSupport/UMDevXS/build must be built.

6.2 Test Instructions

The Driver can be tested with the VAL Test Tool. For the detailed Test Specification refer to *DDK Test Specification* [2]. Refer also to the *README Test Tool VAL* [7] for testing the Driver, which can be found in the TestTools/TestTool_VAL/docs directory.

6.3 Build and Test Instructions for Multiple Configurations

For building and testing multiple or all configurations, go to the TestTools/Regress_vip directory. In this directory, there are scripts for running the regression tests.

Documentation on how to use these scripts is found in the *README Regression System* [6], which can be found in the TestTools/Regress_vip/docs directory.

7 Driver API

7.1 Combined Driver Initialization API

Reference: `Integration/Adapter_DriverInit/incl/api_driver_init.h`

Purpose: Interface to initialize and un-initialize the Combined Driver.

This API contains functions to initialize and un-initialize the Combined Driver. The Driver must be initialized via this API before any of its other functions can be called.

7.2 Global Control Driver Initialization API

Reference: `Integration/Adapter_DriverInit/incl/api_driver_gc_init.h`

Purpose: Interface to initialize and un-initialize the Global Control Driver.

This API contains functions to initialize and un-initialize the Global Control Driver. The Driver must be initialized via this API before any of its other functions can be called.

7.3 VAL Driver Initialization API

Reference: `Integration/Adapter_DriverInit/incl/api_driver_val_init.h`

Purpose: Interface to initialize and un-initialize the VAL Driver.

This API contains functions to initialize and un-initialize the VAL Driver. The Driver must be initialized via this API before any of its other functions can be called.

7.4 Global Control API

Reference: `Integration/Adapter_GlobalControl/incl/api_global_eip130.h`

Purpose: Interface to obtain global status of the device.

This API contains functions to obtain global status information of the hardware.

7.5 VAL API

Reference: `Integration/Adapter_val/incl/api_val.h`

Purpose: Interface to execute *EIP-130* services.

This API contains functions for executing the services that are provided by the *EIP-130* hardware. This means for example that there are functions for symmetric and asymmetric crypto, asset management, secure debug, etc. Note that the VAL API uses the *EIP-130* HW/FW configuration defined in the `Config/cs_eip130.h` file to enable/provide the correct functionality.

7.6 Memory Access API

Reference: `Integration/Adapter_GlobalControl/incl/api_memxs.h`

Purpose: Interface to read and write internal EIP device memory and registers.

This API contains functions for the low-level EIP device internal memory and registers access. This interface should be used for debugging purposes only. It is available in the Global Control Driver for the user-space driver deployment only.

8 Data Structures

This chapter describes the most important data structures used in the *EIP-130* Example Driver. It also shows how they are used.

8.1 Dynamic Resource (DMA) Data Structures

One of the features of the *EIP-130* hardware is its DMA engine that processes data buffers autonomously. The data buffers accessed by *EIP-130* DMA are referred to as dynamic resources. These resources are in the host system memory and are shared between the host processor and *EIP-130* hardware. They are dynamic in the sense that their location in memory is not known in advance. These resources must be allocated at run-time by the host software (such as application and Driver) and assigned addresses that can be used by the host processor and *EIP-130* hardware to access them. Depending on the used OS and MMU hardware the address used by the host processor and the *EIP-130* hardware for the same dynamic resource may be different. In this case, the Driver is responsible for the address translation before the resources is handed over from the host to the *EIP-130* hardware and vice versa.

The internal Driver Framework DMAResource API (see dmares_*.h as well as the *Driver Framework Porting Guide* [1]) defines the way the Driver works with these dynamic resources. Each dynamic resource is identified inside the Driver by a handle (`DMAResource_Handle_t` data type) and is described by its record (`DMAResource_Record_t` data type, see `dmares_record.h`). The implementation of the DMAResource API in the Driver pre-allocates a compile-time configurable number of the DMAResource records during the Driver initialization. Later these records are used by the implementation of the Driver external (available for applications) for the DMA-safe buffer allocation. Each DMA buffer requires one `DMAResource_Record_t` data structure for its description. The `DMAResource_Record_t` data structure contains information about DMA buffer such as requested size, allocated size, alignment, host (virtual) address, DMA (physical) address, and etcetera. A DMAResource record remains internal to the Driver and is not exposed to the application.

8.2 EIP-130 Tokens and Data Buffers

The *EIP-130* hardware processes command tokens that are written to the mailboxes. For every given command token, a result token is returned. A command token can refer to DMA-able data buffers that can hold input data for an operation and/or must be used for the output data of an operation.

When the VAL API is used, the physical use of these tokens is hidden. However, the use of the data buffers depends on the deployment. When the VAL API is used in kernel-space, the provided data buffers must be DMA-safe if bounce buffers use is disabled. When bounce buffers use is enabled or the VAL API is deployed in user-space, the data buffer does not have to be DMA-safe.

9 Driver Configuration Parameters

The Example Drivers have been tested in various configurations. The list below describes important configuration parameters (as compile time parameters).

9.1 Top-level Driver configuration parameters

This section describes the top-level Driver compile-time configuration parameters in the `cs_driver*.h` files.

- DRIVER_NAME
  Specifies the driver name used for reporting in the log files and operating system.

- DRIVER_DEVICE_NAME
  Specifies the device name used for looking up the device handle for the register read and write operations.

- DRIVER_SWAPENDIAN
  The *EIP-130* hardware is little endian. If the host processor is big endian then the endianness conversion shall be enabled in the Driver. This option causes the Driver software to perform any necessary endianness conversion. Note that the *EIP-130* hardware does not support hardware-based byte swapping of words, therefore the DRIVER_ENABLE_SWAP_SLAVE setting cannot be used.

- DRIVER_INTERRUPTS
  The Driver can be used with or without interrupts. It provides API functions that are registered as callbacks to be invoked on receiving interrupts. These callbacks typically work on the mailbox interrupts, like output mailbox full (result token available) or input mailbox empty (command token is processed, and a new command token can be submitted).

  When interrupts are off, the Driver is said to be in “polling mode”. In this mode, the caller of the Driver API should poll the device registers for result token available or if a command token can be submitted.

  When interrupts are enabled, the callbacks are invoked asynchronously with respect to the main application. Special care must be taken to make these callbacks work reliably. If the VAL API is used, the implementation of this API will mostly take care of this.

- DRIVER_BOUNCEBUFFERS
  The *EIP-130* hardware works on DMA buffers in system memory. These buffers must be DMA-safe, i.e. aligned on cache-line size and in the DMA-able address range.

  If the buffer provided by the application is not DMA-safe, the Driver can allocate a DMA-safe buffer internally and copy the result of processing back to the user-provided buffer. Such a buffer allocated by the Driver, is called a “Bounce Buffer”.

  Enabling this parameter configures the Driver to create bounce buffers, which means that the caller does not have to provide DMA-safe buffers.

- DRIVER_PERFORMANCE
  Enabling this parameter in the Driver suppresses informational messages. These messages are helpful when debugging the Driver but otherwise it will slow-down the Driver.

- DRIVER_POLLING_MAXLOOPS
  Maximum number of polling attempts while waiting for some event.

- DRIVER_POLLING_DELAY_MS
  Specifies the polling delay between the two consecutive polling attempts.

- DRIVER_BUFMAN_SWAP_ENABLE
  Enables byte swap for words in DMA buffers, e.g. for the *EIP-130* master interface, for example for the MicroBlaze microcontroller in the Big Endian configuration.

- DRIVER_VEX_STUB_ENABLE
  Enables the Driver VEX Stub integration in the Linux kernel-space driver for proxying the VEX interface to the Linux user-space.

- DRIVER_VEX_PROXY_ENABLE
  Enables the Driver VEX Proxy integration in the Linux user-space driver for the VEX interface proxied to the user-space.

9.2 Top-level Adapter configuration parameters

This section describes the top-level Adapter compile-time configuration parameters in the `cs_adapter*.h` configuration files. The default values of these configuration parameters with full description can be found in the default Adapter configuration files `c_adapter_*.h`. The default configuration file can also contain other parameters not described in this section.

The top-level configuration file is used to override the default values of some configuration parameters. Some of the top-level Adapter configuration parameters are derived from the top-level Driver configuration parameters in the `cs_driver*.h` file and the *EIP-130* HW/FW configuration parameters in the cs_eip130.h file. Note that derived configuration parameters (with some exceptions) are not described here.

- LOG_SEVERITY_MAX
  Trace level (for informational messages) for the Adapter module. Default value is LOG_SEVERITY_WARN (or LOG_SEVERITY_CRITICAL if DRIVER_PERFORMANCE is set). It can be set to LOG_SEVERITY_INFO, which will give more traces but will also slow down the Driver operation.

- ADAPTER_STRICT_ARGS / VAL_STRICT_ARGS / VEX_STRICT_ARGS
  Enables strict argument checking for input parameters in the interface functions, typically enabled when DRIVER_PERFORMANCE is not enabled.

- ADAPTER_MAX_DMARESOURCE_HANDLES
  Specify the maximum number of DMA Resources that can be allocated by the driver.

- ADAPTER_IRQ_MBX*
  Names for the interrupt IRQ's used in the interrupt administration data lookup.

- VAL_REMOVE_*
  Parameters that can be used to remove unused functionality to reduce the overall driver footprint. Typically, these defines are derived from the EIP130 parameters.

- VAL_API_ADD_CRYPTO_OFFICER_FUNCTIONS
  When defined the functions that can only be performed by a Crypto Officer, shall be included. By default, this parameter is defined.

- VEX_TRACE_TOKENS / VEX_TRACE_TOKENS_ERROR
  Enables command and result tokens tracing/debugging. Note that during normal use, this define shall be disabled.

- VEX_ENABLE_FIRMWARE_LOAD
  This parameter defines that the firmware load functionality shall be enabled, which is typically derived from the parameter EIP130_ENABLE_FIRMWARE_LOAD.

- VEX_FIRMWARE_FILE
  Specifies the name of the file that holds the General-Purpose Firmware image. This parameter is only used when the firmware load functionality is enabled. By default, the filename is `firmware_eip130ram.sbi`.

- VEX_MAILBOX_NR
  Specifies the mailbox to use.

- VEX_MAILBOX_LINK_ONLY_ONCE
  Specifies if the mailbox must be linked only once, otherwise the mailbox will be linked for every submitted token and released (unlinked) when the result token is read.

- VEX_CHECK_DMA_WITH_TOKEN_ID
  Specifies the driver to check if the DMA operation is finished. When this parameter is enabled, the driver extents the output buffer with a 32-bit word and instructs the *EIP-130* to write the TokenID to this 32-bit word. The DMA operation is regarded finished when this 32-bit word contains the TokenID.

- VEX_KEYPAIR_PUBKEYSIZE_NOTOKENID
  Specifies that the public key buffer size when generating a key pair, shall not include the TokenID size.

- VEX_DISABLE_TOKENID_WR_KEYPAIR
  Specifies that the WriteTokenID indication when generating a key pair, will never be set. This means that the DMA operation finished check is omitted and replaced with a data not zero check. Additionally, the size for the buffers for the private key key blob and the public key will not include the TokenID size when passed on to the *EIP-130*.

- VEX_ENABLE_HW_FUNCTIONS
  Enables the *EIP-130* hardware status, options, and version functions.

- VEX_CRYPTO_OFFICER_ID
  Specifies the Crypto Officer ID to use.

- VEX_POLLING_SKIP_FIRST_DELAYS
  Specifies the number of polls before the polling delay is enabled.

9.3 Top-level Driver Framework (HWPAL) configuration parameters

This section describes the top-level Driver Framework compile-time configuration parameters in the `cs_hwpal*.h` configuration files. The default values of these configuration parameters with full description can be found in the default Driver Framework configuration files `c_hwpal_lkm.h` (kernel-space specific) and `c_hwpal_device_umdevxs.h` (user-space specific). The default configuration files can also contain other parameters not described in this section.

The top-level configuration file is used to override the default values of some configuration parameters. Some of the top-level Driver Framework configuration parameters are derived from the top-level Driver configuration parameters in the `cs_driver*.h` file. Note that derived configuration parameters (with some exceptions) are not described here.

Hardware platform specific settings can be configured in the `cs_hwpal_ext.h` configuration file. For the x86 and PowerPC architectures, a PCI-based kernel integration is used and for any other architectures, an Open Firmware (Platform) kernel integration is used.

- LOG_SEVERITY_MAX
  Trace level (for informational messages) for the Driver Framework modules. Default value is LOG_SEVERITY_WARN (or LOG_SEVERITY_CRITICAL if DRIVER_PERFORMANCE is set). It can be set to LOG_SEVERITY_INFO, which will give more traces but will also slow down the Driver operation.

- HWPAL_STRICT_ARGS_CHECK
  Enables strict argument checking for the input parameters in all the Driver Framework API functions.

- HWPAL_DEVICES
  List of devices with their names and their addresses ranges. All addresses of all devices accessed by the driver are assumed to lie in a single address range, e.g. a single BAR on a PCI device or a single address range as specified in the Open Firmware device tree. The start and end addresses are specified as byte offsets with respect to the start of that range.

- HWPAL_MAX_DEVICE_NAME_LENGTH
  Maximum number of characters (bytes) that can be used for a device name as used in the HWPAL_DEVICES list of devices.

- HWPAL_DEVICE_MAGIC
  Optional magic number used to validate the device administration data.

- HWPAL_DEVICE_DIRECT_MEMIO
  Enables direct access to memory mapped registers of the device bypassing the OS I/O functions.

- HWPAL_PLATFORM_IRQ_IDX
  Index of the IRQ used to query the physical device interrupt number via the Linux platform kernel API.

- HWPAL_DEVICE_ID and HWPAL_VENDOR_ID
  These are only used for a PCI (or PCIe) device and they specify the device and vendor ID of the device. Note that similar parameters must also be specified in the configuration of the Kernel Support Driver in `cs_umdevxs_ext.h`.

- HWPAL_USE_MSI
  This is only used for a PCI (or PCIe) device. It specifies whether the device must use MSI interrupts. Note that a similar parameter must also be specified in the configuration of the Kernel Support Driver in `cs_umdevxs_ext.h`.

- HWPAL_PLATFORM_DEVICE_NAME
  This is only used for an Open Firmware device. It specifies the name of the device as specified in the Open Firmware device tree. Note that a similar parameter must also be specified in the configuration of the Kernel Support Driver in `cs_umdevxs_ext.h`.

- HWPAL_DEVICE_READ_DELAY_US
  `Device_Read32()` function will busy wait for the specified by this parameter number of microseconds before performing the read operation. This is a workaround for posted-writes implementation of the MMIO register slave interface when too fast write-read sequence does not work.

- HWAL_DMARESOURCE_STRICT_ARGS_CHECKS
  Enables strict argument checking for the input parameters of the DMAResource API functions.

- HWAL_DMA_NRESOURCES
  Specifies the maximum number of DMA resources available. Change ADAPTER_MAX_DMARESOURCE_HANDLES instead.

- HWAL_TRACE_DMARESOURCE_LEAKS
  Keep track of allocation and deallocation of DMA resources and check that at driver unload time all allocated DMA resources are freed.

- HWAL_DMARESOURCE_ALLOC_CACHE_COHERENT
  Specifies that all DMA resources must be allocated in cache-coherent memory, which does not require cache invalidate and flush operations from the CPU.

- HWAL_DMARESOURCE_UNCACHED_MAPPING
  When defined enables use of `ioremap_cache` for mapping memory as non-cached. This is used for static DMA banks.

- HWAL_DMARESOURCE_MINIMUM_CACHE_CONTROL
  Enable this parameter to use the minimal Linux DMA API. This will also cause the Linux Kernel to perform fewer sanity checks on the provided DMA buffers.

- HWAL_DMARESOURCE_DCACHE_LINE_SIZE
  Specifies the size of a single data cache line in bytes.

- HWPAL_DEVICE_COUNT
  Maximum number of devices in the device list must be equal to or greater than the number of devices in HWPAL_DEVICES.

9.3.1 Default Driver Configuration

The Driver has following default configuration set:

- SWAPENDIAN is set depending on the ARCH switch.
- Interrupts Off.
- Bounce Buffers Off.
- Performance On.
- DMA ready check is On.
- Crypto Officer functions are enabled.
- The enabled *EIP-130* HW/FW configuration is taken from the `cs_eip130.h` file. This means that the parameters in this file define the algorithms, schemes and overall functionality used.

For other configurations, refer to the *DDK Test Specification* [2].

9.4 Driver Traces (Logging) Configuration

To get information about what the driver does, the traces can be enabled at a low level. By default, the trace level is warning for the driver. This can be set to the informational level to get as many traces as possible. The driver tracing functionality is implemented in the `Kit/Log` component. The driver traces can be configured in the Adapter layer (`Config/cs_adapter.h`) as well as in the Driver Framework layer (`Config/cs_hwpal.h`).

To enable the informational traces in the driver Adapter layer, ensure that the following is present in the `Config/cs_adapter.h` file:

```
#define LOG_SEVERITY_MAX  LOG_SEVERITY_INFO
```

To enable the informational traces in the Driver Framework layer, ensure that the following is present in the `Config/cs_hwpal.h` file:

```
#define HWPAL_LOG_SEVERITY  LOG_SEVERITY_INFO

#define HWPAL_TRACE_DEVICE_FIND                // Trace Device_Find()
#define HWPAL_TRACE_DEVICE_READ                // Trace Device_Read32()
#define HWPAL_TRACE_DEVICE_WRITE               // Trace Device_Write32()
#define HWPAL_TRACE_DMARESOURCE_WRITE          // Trace DMAResource_Write32()
#define HWPAL_TRACE_DMARESOURCE_READ           // Trace DMAResource_Read32()
#define HWPAL_TRACE_DMARESOURCE_PREPOSTDMA     // Trace DMAResource_PreDMA() and
                                                       DMAResource_PostDMA()
#define HWPAL_TRACE_DMARESOURCE_BUF            // Trace DMAResource_Alloc(),
                                                   DMAResource_Register(),
                                                   DMAResource_Release()
```

Inspect the `cs_adapter.h` and `cs_hwpal.h` files for the details how the traces can be configured. Note how the `DRIVER_PERFORMANCE` configuration parameter influences the tracing level in these files.

Note: Configuring the driver for the informational traces in the Adapter and especially in the Driver Framework layers will result in very long log files produced by the driver when used with applications performing many requests to the driver.

(End of Document)